Chapter 04: Dimension Reduction

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [1]:
```
import matplotlib.pyplot as plt
import mlba
import numpy as np
import pandas as pd
from sklearn import preprocessing
from sklearn.decomposition import PCA
%matplotlib inline
```

```
no display found. Using non-interactive Agg backend
```

Example 1: House Prices in Boston

Table 4.3 Summary Statistics for the Boston Housing Data

We immediately see that the different variables have very different ranges of values. We will soon see how variation in scale across variables can distort analyses if not treated properly. Another observation that can be made is that the mean of the first variable, CRIM (as well as several others), is much larger than the median, indicating right skew. None of the variables have missing values. There also do not appear to be indications of extreme values that might result from typing errors.

In [2]:
```
#VB67
# Loading the Boston Housing dataset using the mlba library
bostonHousing_df = mlba.load_data('BostonHousing.csv')
# Renaming the column 'CAT. MEDV' to 'CAT_MEDV' for easier reference
bostonHousing_df = bostonHousing_df.rename(columns={'CAT. MEDV': 'CAT_MEDV'}
# Displaying the first 9 rows of the dataset to get an initial look at the c
bostonHousing_df.head(9)
# Generating a summary of basic statistics for all numerical variables
bostonHousing_df.describe()

# Computing key statistics for the variable 'CRIM' (per capita crime rate by
print('Mean : ', bostonHousing_df.CRIM.mean())
print('Std. dev : ', bostonHousing_df.CRIM.std())
print('Min : ', bostonHousing_df.CRIM.min())
print('Max : ', bostonHousing_df.CRIM.max())
print('Median : ', bostonHousing_df.CRIM.median())
print('Length : ', len(bostonHousing_df.CRIM))
print('Number of missing values : ', bostonHousing_df.CRIM.isna().sum())

# Computing summary statistics (mean, standard deviation, min, max, median,
pd.DataFrame({'mean': bostonHousing_df.mean(),
              'sd': bostonHousing_df.std(),
              'min': bostonHousing_df.min(),
              'max': bostonHousing_df.max(),
              'median': bostonHousing_df.median(),
              'length': len(bostonHousing_df),
              'miss.val': bostonHousing_df.isna().sum(),
             })
```

```
Mean :  3.613523557312254
Std. dev :  8.60154510533249
Min :  0.00632
Max :  88.9762
Median :  0.25651
Length :  506
Number of missing values :  0
```

Out[2]:

	mean	sd	min	max	median	length	miss.val
CRIM	3.613524	8.601545	0.00632	88.9762	0.25651	506	0
ZN	11.363636	23.322453	0.00000	100.0000	0.00000	506	0
INDUS	11.136779	6.860353	0.46000	27.7400	9.69000	506	0
CHAS	0.069170	0.253994	0.00000	1.0000	0.00000	506	0
NOX	0.554695	0.115878	0.38500	0.8710	0.53800	506	0
RM	6.284634	0.702617	3.56100	8.7800	6.20850	506	0
AGE	68.574901	28.148861	2.90000	100.0000	77.50000	506	0
DIS	3.795043	2.105710	1.12960	12.1265	3.20745	506	0
RAD	9.549407	8.707259	1.00000	24.0000	5.00000	506	0
TAX	408.237154	168.537116	187.00000	711.0000	330.00000	506	0
PTRATIO	18.455534	2.164946	12.60000	22.0000	19.05000	506	0
LSTAT	12.653063	7.141062	1.73000	37.9700	11.36000	506	0
MEDV	22.532806	9.197104	5.00000	50.0000	21.20000	506	0
CAT_MEDV	0.166008	0.372456	0.00000	1.0000	0.00000	506	0

Partial output

Table 4.4 Correlation Table for Boston Housing Data

Next, we summarize relationships between two or more variables. For numerical variables, we can compute a complete matrix of correlations between each pair of variables, using the pandas method corr(). Table 4.4 shows the correlation matrix for the Boston Housing variables. We see that most correlations are low and that many are negative.

In [6]:

```
#VB67
# Showing correlation matrix
bostonHousing_df.corr().round(2)
```

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTR
CRIM	1.00	-0.20	0.41	-0.06	0.42	-0.22	0.35	-0.38	0.63	0.58	
ZN	-0.20	1.00	-0.53	-0.04	-0.52	0.31	-0.57	0.66	-0.31	-0.31	-
INDUS	0.41	-0.53	1.00	0.06	0.76	-0.39	0.64	-0.71	0.60	0.72	
CHAS	-0.06	-0.04	0.06	1.00	0.09	0.09	0.09	-0.10	-0.01	-0.04	
NOX	0.42	-0.52	0.76	0.09	1.00	-0.30	0.73	-0.77	0.61	0.67	
RM	-0.22	0.31	-0.39	0.09	-0.30	1.00	-0.24	0.21	-0.21	-0.29	-
AGE	0.35	-0.57	0.64	0.09	0.73	-0.24	1.00	-0.75	0.46	0.51	
DIS	-0.38	0.66	-0.71	-0.10	-0.77	0.21	-0.75	1.00	-0.49	-0.53	-
RAD	0.63	-0.31	0.60	-0.01	0.61	-0.21	0.46	-0.49	1.00	0.91	
TAX	0.58	-0.31	0.72	-0.04	0.67	-0.29	0.51	-0.53	0.91	1.00	
PTRATIO	0.29	-0.39	0.38	-0.12	0.19	-0.36	0.26	-0.23	0.46	0.46	
LSTAT	0.46	-0.41	0.60	-0.05	0.59	-0.61	0.60	-0.50	0.49	0.54	
MEDV	-0.39	0.36	-0.48	0.18	-0.43	0.70	-0.38	0.25	-0.38	-0.47	
CAT_MEDV	-0.15	0.37	-0.37	0.11	-0.23	0.64	-0.19	0.12	-0.20	-0.27	-

Aggregation and Pivot Tables

Table 4.5 Number of Neighborhoods that Bound the Charles River Vs. Those That Do Not

Another very useful approach for exploring the data is aggregation by one or more variables. For aggregation by a single variable, we can use the pandas method value_counts(). For example, Table 4.5 shows the number of neighborhoods that bound the Charles River vs. those that do not (the variable CHAS is chosen as the grouping variable). It appears that the majority of neighborhoods (471 of 506) do not bound the river.

In [7]:

```
#VB67
# Count the number of occurrences for each category in the CHAS variable
bostonHousing_df = mlba.load_data('BostonHousing.csv')
bostonHousing_df = bostonHousing_df.rename(columns={'CAT. MEDV': 'CAT_MEDV'}
bostonHousing_df.CHAS.value_counts()
```

Out[7]:

```
CHAS
0    471
1     35
Name: count, dtype: int64
```

Table 4.6 Average MEDV by CHAS and RM

The groupby() method can be used for aggregating by one or more variables and computing a range of summary statistics (count, mean, median, etc.). For categorical variables, we obtain a breakdown of the records by the combination of categories. For instance, in Table 4.6, we compute the average MEDV by CHAS and RM. Note that the numerical variable RM (the average number of rooms per dwelling in the neighborhood) should be first grouped into bins of size 1 (0–1, 1–2, etc.). Note the empty values, denoting that there are no neighborhoods in the dataset with those combinations (e.g., bounding the river and having on average three rooms).

In [8]:
```
#VB67
# Group data by RM bins and CHAS, then compute the average MEDV for each gro
bostonHousing_df['RM_bin'] = pd.cut(bostonHousing_df.RM, range(10), labels=F
bostonHousing_df.groupby(['RM_bin', 'CHAS'])['MEDV'].mean()
```

Out[8]:
```
RM_bin  CHAS
3       0       25.300000
4       0       15.407143
5       0       17.200000
        1       22.218182
6       0       21.769170
        1       25.918750
7       0       35.964444
        1       44.066667
8       0       45.700000
        1       35.950000
Name: MEDV, dtype: float64
```

Table 4.7 Pivot Tables in Python

In [9]:
```
#VB67
#loading dataset
bostonHousing_df = mlba.load_data('BostonHousing.csv')
bostonHousing_df = bostonHousing_df.rename(columns={'CAT. MEDV': 'CAT_MEDV'}
# create bins of size 1
bostonHousing_df['RM_bin'] = pd.cut(bostonHousing_df.RM, range(10), labels=F

# use pivot_table() to reshape data and generate pivot table
pd.pivot_table(bostonHousing_df, values='MEDV', index=['RM_bin'], columns=['
               aggfunc='mean', margins=True)
```

Out[9]:

CHAS	0	1	All
RM_bin			
3	25.300000	NaN	25.300000
4	15.407143	NaN	15.407143
5	17.200000	22.218182	17.551592
6	21.769170	25.918750	22.015985
7	35.964444	44.066667	36.917647
8	45.700000	35.950000	44.200000
All	22.093843	28.440000	22.532806

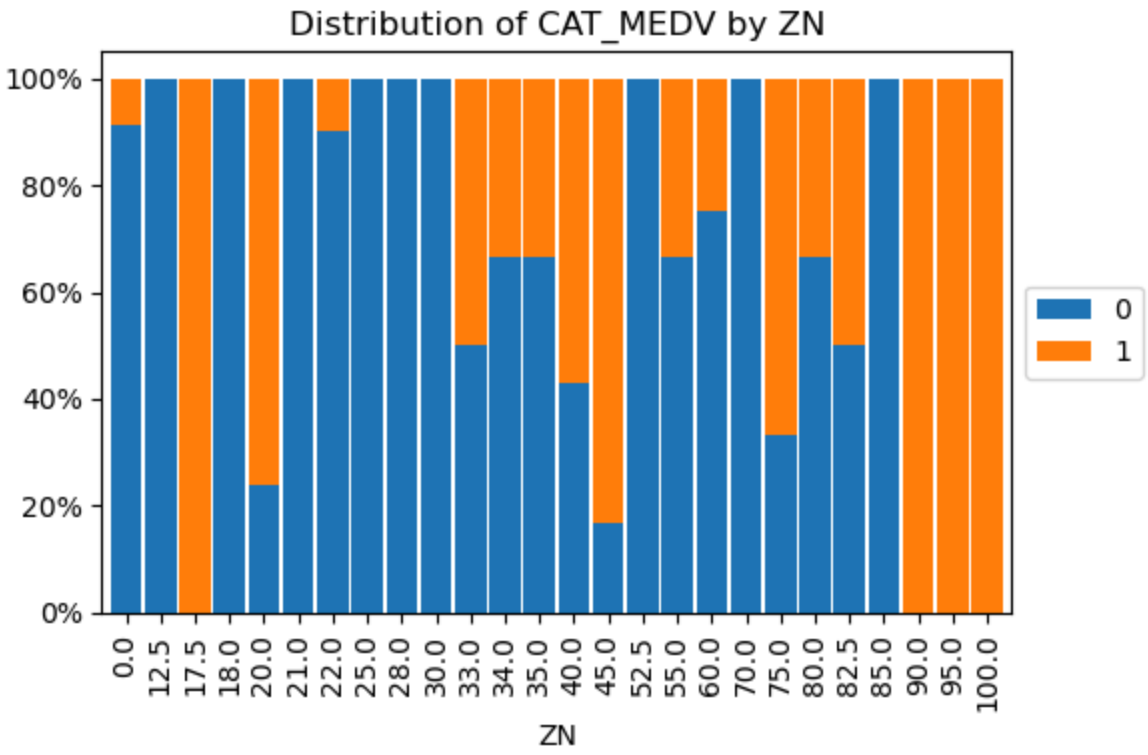
Reducing the Number of Categories in Categorical Variables

In [11]:

```
#VB67
# use method crosstab to create a cross-tabulation of two variables
tbl = pd.crosstab(bostonHousing_df.CAT_MEDV, bostonHousing_df.ZN)

# convert numbers to ratios
propTbl = tbl / tbl.sum()
propTbl.round(2)

# plot the ratios in a stacked bar chart
ax = propTbl.transpose().plot(kind='bar', stacked=True, width=0.9, figsize=(
ax.set_yticks([0, 0.2, 0.4, 0.6, 0.8, 1])
ax.set_yticklabels([f'{x:.0%}' for x in ax.get_yticks()])
plt.title('Distribution of CAT_MEDV by ZN')
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))
plt.tight_layout()
#Display plot
plt.show()
```



Principal Component Analysis

Principal component analysis (PCA) is a useful method for dimension reduction, especially when the number of variables is large. PCA is especially valuable when we have subsets of measurements that are measured on the same scale and are highly correlated. In that case, it provides a few variables (often as few as three) that are weighted linear combinations of the original variables and that retain the majority of the information of the full original set. PCA is intended for use with numerical variables. For categorical variables, other methods such as correspondence analysis are more suitable.

Example 2: Breakfast Cereals

Principal component analysis (PCA) is a useful method for dimension reduction, especially when the number of variables is large. PCA is especially valuable when we have subsets of measurements that are measured on the same scale and are highly correlated. In that case, it provides a few variables (often as few as three) that are weighted linear combinations of the original variables and that retain the majority of the information of the full original set. PCA is intended for use with numerical variables. For categorical variables, other methods such as correspondence analysis are more suitable.

Table 4.11 PCA on the Two Variables Calories and Rating

Running PCA in scikit-learn is done with the class sklearn.decomposition.PCA. Table 4.11 shows the output from running PCA on the two variables calories and rating. The value pca.components_ of this function is the rotation matrix, which gives the weights that are used to project the original points onto the two new directions. The weights for are given by (0.847, 0.532), and for they are given by (0.532, 0.847). The summary gives the reallocated variance: accounts for 86% of the total variability and for the remaining 14%. Therefore, if we drop , we still maintain 86% of the total variability.

In [12]:
```
#VB67
# Loading the Cereals dataset
cereals_df = mlba.load_data('Cereals.csv')
# Creating a PCA object to extract the first 2 principal components
pca = PCA(n_components=2)
# Fitting PCA on the selected numeric columns: calories and rating
pca.fit(cereals_df[['calories', 'rating']])

# Building a summary table of the fitted PCA
pcaSummary = pd.DataFrame({'Standard deviation': np.sqrt(pca.explained_varia
                           'Proportion of variance': pca.explained_variance_
                           'Cumulative proportion': np.cumsum(pca.explained_
pcaSummary = pcaSummary.transpose() # Put the statistics as rows and the PCs
pcaSummary.columns = ['PC1', 'PC2'] # Name the columns
pcaSummary.round(4) # Round for neat presentation
```

Out[12]:

	PC1	PC2
Standard deviation	22.3165	8.8844
Proportion of variance	0.8632	0.1368
Cumulative proportion	0.8632	1.0000

In [13]:
```
#VB67
# Build a table of PCA component weights
pcaComponents_df = pd.DataFrame(pca.components_.transpose(), columns=['PC1',
                                index=['calories', 'rating'])
pcaComponents_df
```

Out[13]:

	PC1	PC2
calories	0.847053	0.531508
rating	-0.531508	0.847053

In [14]:
```
#VB67
# Project each cereal into the PCA space to get "scores"
scores = pd.DataFrame(pca.transform(cereals_df[['calories', 'rating']]),
                      columns=['PC1', 'PC2'])
#show first five rows
scores.head()
```

Out[14]:

	PC1	PC2
0	-44.921528	2.197183
1	15.725265	-0.382416
2	-40.149935	-5.407212
3	-75.310772	12.999126
4	7.041508	-5.357686

Table 4.12 PCA Output Using All 13 Numerical Variables

Note that the first three components account for more than 96% of the total variation associated with all 13 of the original variables. This suggests that we can capture most of the variability in the data with less than 25% of the original dimensions in the data. In fact, the first two principal components alone capture 92.6% of the total variation. However, these results are influenced by the scales of the variables, as we describe next.

In [15]:

```
#VB67
# Creating a PCA object to include all possible principal components
pca = PCA()
# Fitting PCA on all numeric columns starting from the 4th column, after dro
pca.fit(cereals_df.iloc[:, 3:].dropna(axis=0))
# Creating a summary table showing key PCA statistics
pcaSummary_df = pd.DataFrame({'Standard deviation': np.sqrt(pca.explained_va
                              'Proportion of variance': pca.explained_variance_r
                              'Cumulative proportion': np.cumsum(pca.explained_v
# Transpose the table so statistics are in rows and PCs are in columns
pcaSummary_df = pcaSummary_df.transpose()
# Rename columns as PC1, PC2, PC3, etc.
pcaSummary_df.columns = [f'PC{i}' for i in range(1, len(pcaSummary_df.columr
# Round the results to 4 decimal places for cleaner display
pcaSummary_df.round(4)
```

Out[15]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC
Standard deviation	83.7641	70.9143	22.6437	19.1815	8.4232	2.0917	1.6994	0.7796	0.657
Proportion of variance	0.5395	0.3867	0.0394	0.0283	0.0055	0.0003	0.0002	0.0000	0.000
Cumulative proportion	0.5395	0.9262	0.9656	0.9939	0.9993	0.9997	0.9999	1.0000	1.000

In [16]:

```
#VB67
# Creating a DataFrame showing how much each original variable contributes t
pcaComponents_df = pd.DataFrame(pca.components_.transpose(), columns=pcaSumm
```

```
                                    index=cereals_df.iloc[:, 3:].columns)
# Displaying the loadings for the first 5 principal components
pcaComponents_df.iloc[:,:5]
```

Out[16]:

	PC1	PC2	PC3	PC4	PC5
calories	0.077984	-0.009312	0.629206	-0.601021	0.454959
protein	-0.000757	0.008801	0.001026	0.003200	0.056176
fat	-0.000102	0.002699	0.016196	-0.025262	-0.016098
sodium	0.980215	0.140896	-0.135902	-0.000968	0.013948
fiber	-0.005413	0.030681	-0.018191	0.020472	0.013605
carbo	0.017246	-0.016783	0.017370	0.025948	0.349267
sugars	0.002989	-0.000253	0.097705	-0.115481	-0.299066
potass	-0.134900	0.986562	0.036782	-0.042176	-0.047151
vitamins	0.094293	0.016729	0.691978	0.714118	-0.037009
shelf	-0.001541	0.004360	0.012489	0.005647	-0.007876
weight	0.000512	0.000999	0.003806	-0.002546	0.003022
cups	0.000510	-0.001591	0.000694	0.000985	0.002148
rating	-0.075296	0.071742	-0.307947	0.334534	0.757708

Normalizing the Data

A further use of PCA is to understand the structure of the data. This is done by examining the weights to see how the original variables contribute to the different principal components. In our example, it is clear that the first principal component is dominated by the sodium content of the cereal: it has the highest (in this case, positive) weight. This means that the first principal component is in fact measuring how much sodium is in the cereal. Similarly, the second principal component seems to be measuring the amount of potassium. Since both these variables are measured in milligrams, whereas the other nutrients are measured in grams, the scale is obviously leading to this result. The variances of potassium and sodium are much larger than the variances of the other variables, and thus the total variance is dominated by these two variances. A solution is to normalize the data before performing the PCA. Normalization (or standardization) means replacing each original variable by a standardized version of the variable that has unit variance. This is easily accomplished by dividing each variable by its standard deviation. The effect of this normalization is to give all variables equal importance in terms of variability.

When should we normalize the data like this? It depends on the nature of the data. When the units of measurement are common for the variables (e.g., dollars), and when their

scale reflects their importance (sales of jet fuel, sales of heating oil), it is probably best not to normalize (i.e., not to rescale the data so that they have unit variance). If the variables are measured in different units so that it is unclear how to compare the variability of different variables (e.g., dollars for some, parts per million for others) or if for variables measured in the same units, scale does not reflect importance (earnings per share, gross revenues), it is generally advisable to normalize. In this way, the differences in units of measurement do not affect the principal components' weights. In the rare situations where we can give relative weights to variables, we multiply the normalized variables by these weights before doing the PCA.

Table 4.13 PCA Output Using All Normalized 13 Numerical Variables

In [17]:
```
#VB67
# Creating a PCA object to include all principal components
pca = PCA()
# Standardizing the numeric data (mean = 0, std = 1) before fitting PCA for
pca.fit(preprocessing.scale(cereals_df.iloc[:, 3:].dropna(axis=0)))
# Creating a summary table showing key PCA statistics
pcaSummary_df = pd.DataFrame({'Standard deviation': np.sqrt(pca.explained_va
                           'Proportion of variance': pca.explained_variance_r
                           'Cumulative proportion': np.cumsum(pca.explained_v
# Transpose the table so statistics appear as rows and PCs as columns
pcaSummary_df = pcaSummary_df.transpose()
# Renaming columns to represent each principal component (PC1, PC2, etc.)
pcaSummary_df.columns = [f'PC{i}' for i in range(1, len(pcaSummary_df.columr
# Round the results to 4 decimal places for a cleaner display
pcaSummary_df.round(4)
```

Out[17]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
Standard deviation	1.9192	1.7864	1.3912	1.0166	1.0015	0.8555	0.8251	0.6496	0.5658
Proportion of variance	0.2795	0.2422	0.1469	0.0784	0.0761	0.0555	0.0517	0.0320	0.0243
Cumulative proportion	0.2795	0.5217	0.6685	0.7470	0.8231	0.8786	0.9303	0.9623	0.9866

In [18]:
```
#VB67
# Creating a DataFrame of PCA loadings showing how each original variable co
pcaComponents_df = pd.DataFrame(pca.components_.transpose(), columns=pcaSumm
                                index=cereals_df.iloc[:, 3:].columns)
# Displaying the loadings for the first 5 principal components
pcaComponents_df.iloc[:,:5]
```

Out[18]:

	PC1	PC2	PC3	PC4	PC5
calories	-0.299542	0.393148	0.114857	-0.204359	0.203899
protein	0.307356	0.165323	0.277282	-0.300743	0.319749
fat	-0.039915	0.345724	-0.204890	-0.186833	0.586893
sodium	-0.183397	0.137221	0.389431	-0.120337	-0.338364
fiber	0.453490	0.179812	0.069766	-0.039174	-0.255119
carbo	-0.192449	-0.149448	0.562452	-0.087835	0.182743
sugars	-0.228068	0.351434	-0.355405	0.022707	-0.314872
potass	0.401964	0.300544	0.067620	-0.090878	-0.148360
vitamins	-0.115980	0.172909	0.387859	0.604111	-0.049287
shelf	0.171263	0.265050	-0.001531	0.638879	0.329101
weight	-0.050299	0.450309	0.247138	-0.153429	-0.221283
cups	-0.294636	-0.212248	0.140000	-0.047489	0.120816
rating	0.438378	-0.251539	0.181842	-0.038316	0.057584

In [ ]: